Webcam to 3D Motion Grid

CS 4143 Computer Graphics Final Project Report

Charlie Autry

Project Description

The project is a real-time graphics application that turns a live webcam feed into an interactive 3D scene. Every cell of a configurable grid (default 64x48) is rendered as a 3D column whose color is sampled from the corresponding webcam pixel and whose height is driven by motion detected in the camera frame. When something moves in front of the camera, the columns underneath that motion extrude upward, like a 3D relief that responds to whatever is happening in the room. The user can orbit around the grid with the mouse, zoom in and out, pan, switch between color and monochrome modes, mirror the feed, or change the grid resolution on the fly.

The original inspiration came from the "Webcam Pixel Grid" component on Aceternity UI, which is a 2D web canvas effect. I wanted to see what it would look like rendered as actual 3D geometry instead, with proper lighting and a camera you can move around.

Details of Implementation

The application is C++ using OpenGL 3.3 core profile through GLFW for windowing, GLM for math, and OpenCV 4 for camera capture and motion detection. Rendering goes through instanced draw calls: a single unit cube mesh is drawn once per frame for every cell in the grid (3,072 cubes at default resolution), with per-instance attributes pushed to the GPU as a separate buffer (grid position, RGB color, extrusion height). The whole grid is one glDrawElementsInstanced call regardless of grid size.

Each frame, OpenCV captures from the default webcam, downsamples the frame to grid resolution for color sampling, and runs dense Farneback optical flow at 320x240 against the previous frame to extract motion. The motion magnitudes are downsampled to grid resolution and used as the target heights for each column. Heights are not snapped directly: they lerp toward the target with an asymmetric rate (fast rise, slow fall) so the columns trail and decay naturally instead of popping. The smoothed heights and color values are uploaded to the GPU using buffer orphaning so the upload does not stall the draw call.

Shading is Phong (ambient + diffuse + specular). The light position sits above and forward of the grid center, and its intensity is modulated by the average motion across the grid: when there is lots of movement the scene brightens, when nothing is happening it dims. The orbit camera uses spherical coordinates (yaw, pitch, distance) around a target point, with mouse callbacks wired into GLFW for left-drag rotate, right-drag pan, and scroll-zoom.

For the HUD I wrote a small text renderer using an 8x8 bitmap font baked into a single texture atlas. It draws FPS, current mode, height multiplier, sensitivity, grid resolution, and the controls hint at the bottom. The atlas is built procedurally at startup from packed glyph data.

Challenges and Solutions

The biggest hurdle was getting the motion detection to feel right. The original plan was simple frame differencing with cv::absdiff on consecutive grayscale frames, but that ended up jittery and binary: every column would either snap to full height or stay flat, and any tiny bit of camera noise triggered spurious extrusion. I switched to Farneback dense optical flow, which gives a continuous motion magnitude per pixel and produces a much smoother result. Running it at grid resolution directly was unstable because the flow algorithm needs enough pixels to find good correspondences, so I run it at 320x240 and then downsample the magnitude map to grid resolution with INTER_AREA, which preserves motion energy across the downsample.

Even with optical flow, columns popping straight to their target heights felt bad. A single lerp rate either looked too snappy or too laggy depending on the value. Splitting it into a rise speed and a fall speed fixed it: fast rise so the columns react immediately, slow fall so the motion leaves a soft trail.

Performance was a concern at higher grid resolutions. At 128x96 there are 12,288 columns and drawing them one by one was not going to work. Instanced rendering brought it down to a single draw call, and using buffer orphaning (calling glBufferData with nullptr before glBufferSubData) avoids the GPU pipeline stalling waiting on the previous frame's upload to finish.

Build setup was a smaller but real challenge. The starter framework from class had its own loader, math library, and asset pipeline that I did not end up needing once I switched to GLFW + GLM + OpenCV. Wiring up CMake to find OpenCV and link everything cleanly without dragging in unused parts took some iteration, especially since OpenCV's Windows install is not really meant to be embedded into a CMake project.

Reflection

This was the most fun assignment I have done in the class, mostly because the visual feedback loop is so immediate: I could change the lerp rate or the lighting parameters and instantly see the difference on screen with my own face. It also pulled together a lot of what we covered during the semester (rendering pipelines, transformation matrices, Phong lighting, procedural geometry), and I had to actually understand each piece rather than just paste a working setup.

The thing I would most want to revisit if I had more time is replacing the per-frame buffer upload with persistent mapped buffers, since that upload is technically still the heaviest single operation in the loop. I would also like to push the optical flow plus height smoothing onto the GPU as a compute shader so the whole motion pipeline lives there. Both felt out of scope for the deadline.

The biggest takeaway is how much performance work in graphics is really about avoiding stalls and CPU-to-GPU round trips. Once I had instanced rendering and buffer orphaning in place, the bottleneck became OpenCV (specifically the optical flow pass), not the actual rendering, even at 12k columns. That surprised me and turned my mental model of "GPU equals slow" upside down.